



#	سوالات	بارم												
۱	مفهوم RDF Triple را تعریف کنید و اجزای آن Subject، Predicate، Object را با یک مثال از داده دانشگاهی (مثل جدول ۸.۲۸) توضیح دهید. چه تفاوت کلیدی با مدل E-R دارد؟													
۲	پنج تراکنش زیر در یک پایگاه داده روی داده‌های W, X, Y, Z اجرا می‌شوند: $T_0: R_0(W), W_0(X), W_0(Z)$ $T_1: R_1(X), W_1(Y)$ $T_2: W_2(Z), W_2(V)$ $T_3: R_3(V), R_3(X)$ $T_4: W_4(W), R_4(Y)$ دستورات به ترتیب زمان اجرا (نه لزوماً به ترتیب نوشته‌شده در بالا) به شکل زیر رخ داده‌اند: عملیات گام 1 $W_4(W)$ 2 $R_0(W)$ 3 $R_4(Y)$ 4 $W_0(X)$ 5 $R_1(X)$ 6 $W_0(Z)$ 7 $W_1(Y)$ 8 $W_2(Z)$ 9 $W_2(V)$ 10 $R_3(V)$ 11 $R_3(X)$ الف) با استفاده از گراف پیش‌نیازی (Precedence Graph)، بررسی کنید این زمان‌بندی Conflict Serializable است یا نه. ب) تمام یال‌های گراف را با ذکر داده و نوع تعارض (RW/WR/WW) مشخص کنید. ج) در صورت سریال‌پذیر بودن، همه ترتیب‌های سریال معادل ممکن را بیابید (توجه: ممکن است بیش از یک پاسخ درست باشد).													
۳	سه تراکنش T_1, T_2, T_3 روی داده‌های A و B اجرا می‌شوند: زمان‌بند S به این ترتیب اجرا شده است: <table><tr><th>گام</th><th>T_3</th><th>T_2</th><th>T_1</th></tr><tr><td>1</td><td></td><td></td><td>Read(A)</td></tr><tr><td>2</td><td></td><td>Write(A)</td><td></td></tr></table>	گام	T_3	T_2	T_1	1			Read(A)	2		Write(A)		
گام	T_3	T_2	T_1											
1			Read(A)											
2		Write(A)												

A:=A-50 Write(A)			3
	Read(B)		4
Read(B)			5
B:=B+30 Write(B)			6
		Write(A)	7
	commit		8
commit			9
		commit	10

مطلوب است:

الف) آیا زمان‌بند S از نظر Conflict Serializable بودن، سریال‌پذیر تعارضی است؟ (با رسم گراف پیش‌نیازی)

ب) آیا S از نظر View Serializable بودن، سریال‌پذیر دیدگاهی است؟ هر سه شرط تعادل دیدگاهی را به‌طور جداگانه روی A و B بررسی کنید.

ج) نتیجه نهایی را با ذکر دقیق دلیل رد یا قبول توضیح دهید.

۴ پنج تراکنش با داده‌های A، B، C، D، E روی این دنباله از درخواست‌ها اجرا می‌شوند: $(TS(T_1)=1 \dots TS(T_5)=5)$

گام	عملیات
1	$T_4: \text{read}(A)$
2	$T_2: \text{read}(B)$
3	$T_1: \text{read}(B)$
4	$T_3: \text{write}(B)$
5	$T_2: \text{read}(C)$
6	$T_4: \text{read}(C)$
7	$T_1: \text{read}(A)$
8	$T_5: \text{read}(D)$
9	$T_1: \text{commit}$
10	$T_2: \text{commit}$

T ₄ : write(E)	11
T ₅ : write(D)	12
T ₅ : commit	13
T ₃ : write(C)	14
T ₃ : commit	15

مطلوب است:

الف) با اعمال پروتکل Wound-Wait ، در هر گام مشخص کنید قفل دومرحله rigorous گرفته می شود، تراکنش منتظر می ماند، یا تراکنش دیگری rollback می شود.

ب) مشخص کنید کدام تراکنش(ها) rollback می شوند و دقیقاً بعد از کدام درخواست.

ج) نشان دهید در این مثال چگونه از وقوع بن بست (Deadlock) جلوگیری می کند(چطور به بن بست می خورد و این مانع آن می شود).

۵ یک شبکه ی وب ساده با چهار صفحه ی وب A ، B ، C و D را در نظر بگیرید. هر صفحه در ابتدا دارای امتیاز PageRank برابر با ۰.۲۵ (۱/۴) است. اگر امتیازهای PageRank را به صورت تکراری محاسبه کنیم، پس از اولین تکرار، امتیاز PageRank صفحه ی P[a] چه خواهد بود؟

$$P = \begin{pmatrix} 0.25 \\ 0.25 \\ 0.25 \\ 0.25 \end{pmatrix} \quad T = \begin{pmatrix} 0 & 0.5 & 0.5 & 0 \\ 0.5 & 0 & 0.5 & 0 \\ 0.5 & 0.5 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

موفق باشید-تقی نژاد



جواب ۱:

طبق اسلایدهای ۸.۲۴ تا ۸.۲۵:

یک عبارت (RDF (RDF statement) از سه بخش تشکیل شده که به آن **triple** می‌گویند:۱. **Subject**: منبعی که توسط **triple** توصیف می‌شود.۲. **Predicate**: رابطه بین **subject** و **object** را نشان می‌دهد (می‌تواند **attribute-name** یا **relationship-name** باشد).۳. **Object**: منبعی که به **subject** مرتبط است (یا یک مقدار).

فرمت کلی (ID, attribute-name, value) یا (ID1, relationship-name, ID2)

مثال از داده دانشگاهی:

10101 name "Srinivasan"

10101 inst_dept comp_sci

sec1 sec_course CS-101

تفاوت با مدل **E-R**: برخلاف مدل **E-R** که از روابط **n-ary** (چندتایی) پشتیبانی می‌کند، مدل **RDF** فقط از روابط دوتایی (**binary relationships**) پشتیبانی می‌کند وروابط **n-ary** پیچیده‌تر را نمی‌تواند به‌طور مستقیم نمایش دهد. همچنین هر **triple** دارای یک شناسه یکتا به نام **IRI (International Resource Identifier)** است.

جواب ۲

حل کامل سؤال

گام ۱: شناسایی تعارض‌ها بر اساس هر داده

W: داده• $R_0(W) : W_4(W)$ گام ۱• $T_4 \rightarrow T_0$ یا $WR \rightarrow$ نوع W_4-R_0 : تعارض**Y:** داده• $W_1(Y) : R_4(Y)$ گام ۳• $T_4 \rightarrow T_1$ یا $RW \rightarrow$ نوع R_4-W_1 : تعارض**X:** داده• $R_3(X) : R_1(X)$ گام ۱۱، $W_0(X)$ گام ۴



- $W_0-R_1 \rightarrow$ نوع $WR \rightarrow T_0 \rightarrow T_1$ یال
- $W_0-R_3 \rightarrow$ نوع $WR \rightarrow T_0 \rightarrow T_3$ یال
- هر دو خواندن هستند $\rightarrow$ بدون تعارض: R_1-R_3

Z: داده

- $W_2(Z)$ ، گام ۸: $W_0(Z)$ گام ۶
- $T_0 \rightarrow T_2$ یال $WW \rightarrow$ نوع W_0-W_2 : تعارض

V: داده

- $R_3(V)$ ، گام ۱۰: $W_2(V)$ گام ۹
- $T_2 \rightarrow T_3$ یال $WR \rightarrow$ نوع W_2-R_3 : تعارض

گام ۲: جدول یال‌های گراف

نوع تعارض داده یال

$T_4 \rightarrow T_0$	W	WR
$T_4 \rightarrow T_1$	Y	RW
$T_0 \rightarrow T_1$	X	WR
$T_0 \rightarrow T_3$	X	WR
$T_0 \rightarrow T_2$	Z	WW
$T_2 \rightarrow T_3$	V	WR

(Cycle) گام ۳: بررسی وجود دور

با بررسی گراف

$T_4 \rightarrow T_0 \rightarrow T_1$

$T_4 \rightarrow T_1$

$T_0 \rightarrow T_2 \rightarrow T_3$

$T_0 \rightarrow T_3$

است (DAG) هیچ مسیر بازگشتی‌ای به هیچ گره‌ای وجود ندارد $\rightarrow$ گراف بدون دور

است Conflict Serializable نتیجه: زمان‌بندی ☒

گام ۴: استخراج محدودیت‌های ترتیب (Topological Constraints)



از یال ها

- باشد T_1 و T_0 باید قبل از T_4
- باشد T_3 و T_2 و T_1 باید قبل از T_0
- باشد T_3 باید قبل از T_2

است، ترتیب دو تراکنش اول همیشه ثابت است T_4 از T_0 وارد نمی شود و تنها یال وارد به T_4 چون هیچ یالی به

 $T_4, T_0, \dots$

(آزاد است T_3 و T_2 نسبت به T_1) است $T_2 < T_3$ باقی می ماند که تنها قید بین شان T_1, T_2, T_3 پس از آن، سه تراکنش

گام ۵: تمام ترتیب های سریال معادل ممکن

ترتیب سریال معادل

۱ T_4, T_0, T_1, T_2, T_3

۲ T_4, T_0, T_2, T_1, T_3

۳ T_4, T_0, T_2, T_3, T_1

هر سه ترتیب بالا با گراف پیش نیازی سازگارند و پاسخ صحیح محسوب می شوند

هستند (Conflict Equivalent)

جواب ۳: پاسخ (راه حل)

گام ۱: بررسی Conflict Serializability

تعارض های روی **A** ترتیب عملیات $R_1(A)@1, W_2(A)@2, W_1(A)@3, W_3(A)@7$:

یال	نوع	جفت متعارض
$T_1 \rightarrow T_2$	RW	$R_1(A)@1 \rightarrow W_2(A)@2$
$T_2 \rightarrow T_1$	WW	$W_2(A)@2 \rightarrow W_1(A)@3$
$T_1 \rightarrow T_3$	WW	$W_1(A)@3 \rightarrow W_3(A)@7$
$T_2 \rightarrow T_3$	WW	$W_2(A)@2 \rightarrow W_3(A)@7$

تعارض های روی **B** ترتیب عملیات $R_1(B)@5, R_2(B)@4, W_1(B)@6$:



بال نوع جفت متعارض

$R_2(B)@4 \rightarrow W_1(B)@6$ RW $T_2 \rightarrow T_1$

$R_1(B)@5 \rightarrow W_1(B)@6$ (هم تراکنش) — —

گراف پیش‌نیازی نهایی:

$$T_1 \rightarrow T_2, T_2 \rightarrow T_1, T_1 \rightarrow T_3, T_2 \rightarrow T_3$$

بین T_1 و T_2 دور (Cycle) وجود دارد و $(T_1 \rightarrow T_2)$ و $(T_2 \rightarrow T_1)$

نتیجه S: از نظر Conflict Serializable سریال‌پذیر تعارضی نیست.

گام ۲: بررسی View Serializability

طبق تعریف، دو زمان‌بند S و S' View Equivalent هستند اگر و فقط اگر هر سه شرط زیر برقرار باشد:

- شرط ۱ (خواندن اولیه یکسان): اگر T_i در S مقدار اولیه X را می‌خواند، باید در S' هم همین‌طور باشد.
- شرط ۲ (رابطه reads-from یکسان): اگر T_i در S مقدار X نوشته‌شده توسط T_j را می‌خواند، باید در S' هم دقیقاً همین رابطه برقرار باشد.
- شرط ۳ (نوشتن نهایی یکسان): اگر T_i در S آخرین نویسنده X باشد، باید در S' هم آخرین نویسنده X باشد.

بررسی روی داده A

- شرط ۱: فقط $R_1(A)$ وجود دارد که مقدار اولیه A را می‌خواند (چون قبل از آن هیچ نوشتنی روی A رخ نداده $\Rightarrow$ در هر ترتیب سریال معادل، T_1 باید قبل از هر نویسنده دیگری A یعنی T_2 و T_3 بیاید).
- شرط ۲: آخرین نویسنده A در S ، T_3 است ($W_3(A)@7$) آخرین نوشتن روی A است $\Rightarrow$ در ترتیب سریال معادل، T_3 باید آخرین نویسنده A باشد.

نتیجه: تنها ترتیبی که هر دو شرط بالا را ارضا می‌کند $T_1 \rightarrow T_2 \rightarrow T_3$:

بررسی روی داده B تحت همین ترتیب فرضی $(T_1 \rightarrow T_2 \rightarrow T_3)$

- در S ، $R_2(B)@4$ مقدار اولیه B را می‌خواند (چون در گام ۴، هنوز هیچ $Write(B)$ رخ نداده $W_1(B)$ — در گام ۶ اتفاق می‌افتد، بعد از خواندن T_2).
- اما اگر ترتیب سریال $T_1 \rightarrow T_2 \rightarrow T_3$ را اجرا کنیم T_1 به‌طور کامل (شامل $Write(B)$) اجرا می‌شود و سپس T_2 اجرا می‌شود؛ بنابراین در این ترتیب سریال، T_2 مقدار B نوشته‌شده توسط T_1 را می‌خواند، نه مقدار اولیه B .

⚠ این دقیقاً نقض شرط ۱ (و در نتیجه شرط ۲) است: رابطه reads-from برای B در S (چیزی نمی‌خواند / می‌خواند مقدار اولیه) با رابطه reads-from در ترتیب سریال (T_2) $T_1 \rightarrow T_2 \rightarrow T_3$ از T_1 می‌خواند (متفاوت است).

گام ۳: نتیجه‌گیری

- تنها ترتیب سریال ممکن که شرایط مربوط به داده A را ارضا می‌کند، $T_1 \rightarrow T_2 \rightarrow T_3$ است (چون T_1 باید اول باشد و T_3 باید آخر باشد).
- اما همین ترتیب، در بررسی داده B شکست می‌خورد، چون رابطه reads-from برای B را به هم می‌زند.



- از آنجا که هیچ ترتیب سریال دیگری برای ارضای شروط روی A وجود ندارد (T_1) الزاماً اول و T_3 الزاماً آخر است، و این یگانه گزینه روی B شکست می خورد:

نتیجه نهایی: زمان بند S نه Conflict Serializable است و نه View Serializable. ❌

جدول خلاصه شروط

نتیجه روی داده B	روی داده A	شرط
❌	❌	در $T_1 \rightarrow T_2 \rightarrow T_3$ نقض می شود ☒ فقط با T_1 اول
❌	❌	T_2 باید مقدار اولیه بخواند، نه از T_1 ☒ چون کسی از کسی نمی خواند، فقط اولیه (رابطه) reads-from شرط ۲)
☒ (T_1)	☒ تنها نویسنده B است	☒ فقط با T_3 آخر
		نوشتن نهایی (شرط ۳)

۴ پاسخ (راه حل)

پاسخ (راه حل گام به گام)

جدول وضعیت قفل ها را دنبال می کنیم ($S =$ قفل اشتراکی برای read ، $X =$ قفل انحصاری برای write)

تصمیم	وضعیت قفل مورد نیاز عملیات	گام
☒ $S(A)$ به T_4 داده می شود	آزاد	1 $T_4:$ $\text{read}(A)$
☒ $S(B)$ به T_2 داده می شود	آزاد	2 $T_2:$ $\text{read}(B)$
☒ سازگار $(S-S) \Rightarrow S(B)$ به T_1 هم داده می شود	S توسط T_2	3 $T_1:$ $\text{read}(B)$
$T_3(3)$ نسبت به هر دوی T_1 و T_2 جوان تر است $\Rightarrow T_3 \Rightarrow$ صبر می کند (Wait) روی B	S توسط $T_1(1)$ ، $T_2(2)$	4 $T_3:$ $\text{write}(B)$
☒ $S(C)$ به T_2	آزاد	5 $T_2:$ $\text{read}(C)$
☒ سازگار $(S-S) \Rightarrow S(C)$ به T_4 هم	S توسط T_2	6 $T_4:$ $\text{read}(C)$
☒ سازگار $(S-S)$	S توسط T_4	7 $T_1:$ $\text{read}(A)$



تصمیم	وضعیت قفل مورد نیاز	عملیات	گام
$T_5 \checkmark S(D)$	آزاد	T_5 : read(D)	8
قفل‌های A و B توسط T_1 آزاد می‌شوند	—	T_1 : commit	9
قفل‌های B و C توسط T_2 آزاد می‌شوند $\Rightarrow$ حالا B آزاد است $\Rightarrow$ درخواست معلق T_3 (گام ۴) اکنون برآورده می‌شود $X(B)$: به T_3 داده می‌شود	—	T_2 : commit	10
$T_4 \checkmark X(E)$	آزاد	T_4 : write(E)	11
$\checkmark$ ارتقای قفل (Lock Upgrade) بدون تعارض؛ T_5 به $X(D)$	خود T_5 روی D قفل دارد	T_5 : write(D)	12
قفل‌های D توسط T_5 آزاد می‌شود	—	T_5 : commit	13
$T_3(3)$ نسبت به $T_4(4)$ قدیمی‌تر است $\Rightarrow T_3$ ، T_4 را زخمی می‌کند $\Rightarrow$ Rollback T_4 بعد از درخواست write(C) توسط $T_4(4)$ S	T_3 : write(C)	T_3 : write(C)	14
در گام ۱۴؛ سپس $X(C)$ به T_3 داده می‌شود	—	T_3 : commit	15

نکته: چون T_4 در گام ۱۴ rollback می‌شود، قفل $X(E)$ که در گام ۱۱ گرفته بود نیز آزاد می‌شود و هر عملیات بعدی T_4 (که در این زمان بندی دیگر وجود نداشت) لغو می‌شود.

نتیجه‌گیری نهایی

- فقط T_4 rollback می‌شود — دقیقاً بعد از درخواست write(C) در گام ۱۴، چون T_3 (قدیمی‌تر) قفل C را که در تصرف T_4 (جوان‌تر) بود درخواست کرد و طبق قاعدهٔ Wound، T_4 را زخمی rollback/کرد.
- T_4 با همان $TS(T_4)=4$ دوباره از ابتدا اجرا می‌شود. (Restart)
- بقیهٔ تراکنش‌ها T_1 ، T_2 ، T_3 ، T_5 با موفقیت commit می‌شوند.

چرا این مثال بن‌بست را نشان می‌دهد و حل می‌کند؟

اگر Wound-Wait اعمال نمی‌شد و فقط از یک الگوریتم صبر کردن ساده (مثل ۲ PL بدون پیش‌گیری) استفاده می‌شد:

- در گام ۱۱، T_4 منتظر چیزی نبود (E آزاد بود)، اما اگر ترتیب کمی متفاوت بود و T_4 به جای گرفتن E، منتظر قفل C می‌ماند در حالی که T_3 هم منتظر قفلی از T_4 بود، یک چرخهٔ انتظار (Wait-For Cycle) شکل می‌گرفت $T_3 \rightarrow T_4 \rightarrow T_3$ ؛ یعنی بن‌بست.
- اما در Wound-Wait، چون قاعدهٔ «قدیمی هرگز صبر نمی‌کند» برقرار است، در گام ۱۴ به جای اینکه T_3 صبر کند و احتمال بن‌بست ایجاد شود، T_3 (قدیمی‌تر) بلافاصله T_4 (جوان‌تر) را wound می‌کند. این دقیقاً مکانیزمی است که Wound-Wait را «Deadlock-Free» می‌سازد: تراکنش‌های قدیمی‌تر هرگز پشت تراکنش‌های جوان‌تر معطل نمی‌مانند و در عوض آن‌ها را قربانی (rollback) می‌کنند.



جواب ۵

محاسبه $P[A]$

۲: محاسبه ی مجموع سهم لینک های ورودی

$$\sum_{i=1}^4 T[i, A] \cdot P[i] = (0 \times 0.25) + (0.5 \times 0.25) + (0.5 \times 0.25) + (0 \times 0.25) \\ = 0 + 0.125 + 0.125 + 0 = 0.25$$

$\delta = 0.15$ و $N = 4$:

$$P[A] = \frac{0.15}{4} + (1 - 0.15) \times 0.25$$

$$P[A] = 0.0375 + (0.85 \times 0.25)$$

$$P[A] = 0.0375 + 0.2125$$